Vel Tech Group of Institutions

Name: Gontu Yaswanth
Email: vtu33434@veltech.edu.in
Roll no: Vtu33434
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 10_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 32.5

Section 1 : Coding

1. Problem Statement

Sai has been studying various algorithms related to heaps. Recently, he came across a problem where he needed to filter elements from a given array based on certain criteria related to heap properties.

He wants to build a max heap from a given array of integers, delete elements that fall outside the specified range, and then rearrange the elements in the form of a Max Heap.

Write a program to help Sai in implementing the program.

Input Format

The first line of input consists of an integer N, representing the number of elements in the array.

The second line consists of N space-separated integers, representing the elements of the array.

The third line consists of two integers, a and b, separated by a space, representing the range to filter the elements.

Output Format

The output prints the space-separated integers, representing the elements that remain in the max heap after removing elements outside the given range.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
5 7 8 3 12
1 6

Output: 5 3

Answer

```
// You are using GCC
#include <stdio.h>

// heapify function
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;
```

```
        heapify(arr, n, largest);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[20];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int a, b;
    scanf("%d %d", &a, &b);

    int filtered[20], k = 0;

    // filter elements in range [a, b]
    for (int i = 0; i < n; i++) {
        if (arr[i] >= a && arr[i] <= b) {
            filtered[k++] = arr[i];
        }
    }

    // build max heap
    for (int i = k/2 - 1; i >= 0; i--) {
        heapify(filtered, k, i);
    }

    // print result
    for (int i = 0; i < k; i++) {
        printf("%d ", filtered[i]);
    }

    return 0;
}
```

Status : Partially correct ***Marks : 2.5/10***

2. Problem Statement

Maxwell, an enthusiastic computer science student, is exploring the fascinating world of data structures. His task is to create a program that constructs a min heap from a list of integers and identifies negative numbers within it.

Help Maxwell in completing the program.

Input Format

The first line of input consists of an integer N, representing the number of integers in the array.

The second line consists of N space-separated integers, representing the elements of the list.

Output Format

The first line of output prints the min-heap as a space-separated list of integers.

The second line prints the negative numbers, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 7
15 -5 20 -19 23 42 29
Output: -19 -5 20 15 23 42 29
-19 -5

Answer

```
// You are using GCC
#include <stdio.h>

// heapify for min heap
void heapify(int arr[], int n, int i) {
    int smallest = i;
    int left = 2*i + 1;
```

```
    int right = 2*i + 2;

    if (left < n && arr[left] < arr[smallest])
        smallest = left;

    if (right < n && arr[right] < arr[smallest])
        smallest = right;

    if (smallest != i) {
        int temp = arr[i];
        arr[i] = arr[smallest];
        arr[smallest] = temp;

        heapify(arr, n, smallest);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int arr[20];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // build min heap
    for (int i = n/2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }

    // print heap
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // print negative numbers
    for (int i = 0; i < n; i++) {
        if (arr[i] < 0) {
            printf("%d ", arr[i]);
```

```
    }
  }

  return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

You are given an array of n integers. You need to perform q queries of two types:

Q l r — Find the minimum value in the subarray from index l to r (inclusive).U idx val — Update the element at index idx to val.

Implement an efficient solution using a Segment Tree to handle both query and update operations.

Input Format

The first line contains two space-separated integers n and q, representing the size of the array and the number of queries

The second line contains n space-separated integers representing the elements of the array.

Next q lines: Each line contains a query in one of the following formats:

- Q l r
- U idx val

Output Format

For each query of type Q, print the minimum value in the specified range on a new line.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 3 2
4 2 5
Q 0 2
Q 1 1

Output: 2
2

Answer

```
// You are using GCC
#include <stdio.h>

int tree[100];
int arr[20];

// build tree
void build(int node, int start, int end) {
    if (start == end) {
        tree[node] = arr[start];
    } else {
        int mid = (start + end) / 2;
        build(2*node, start, mid);
        build(2*node+1, mid+1, end);

        // store minimum
        tree[node] = (tree[2*node] < tree[2*node+1]) ? tree[2*node] : tree[2*node+1];
    }
}

// query min
int query(int node, int start, int end, int l, int r) {
    if (r < start || end < l) {
        return 100000; // large value
    }

    if (l <= start && end <= r) {
        return tree[node];
    }

    int mid = (start + end) / 2;
    int left = query(2*node, start, mid, l, r);
    int right = query(2*node+1, mid+1, end, l, r);
```

```
    return (left < right) ? left : right;
}

// update value
void update(int node, int start, int end, int idx, int val) {
    if (start == end) {
        arr[idx] = val;
        tree[node] = val;
    } else {
        int mid = (start + end) / 2;

        if (idx <= mid)
            update(2*node, start, mid, idx, val);
        else
            update(2*node+1, mid+1, end, idx, val);

        tree[node] = (tree[2*node] < tree[2*node+1]) ? tree[2*node] : tree[2*node+1];
    }
}

int main() {
    int n, q;
    scanf("%d %d", &n, &q);

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    build(1, 0, n-1);

    while (q--) {
        char type;
        scanf(" %c", &type);

        if (type == 'Q') {
            int l, r;
            scanf("%d %d", &l, &r);
            printf("%d\n", query(1, 0, n-1, l, r));
        } else if (type == 'U') {
            int idx, val;
            scanf("%d %d", &idx, &val);
            update(1, 0, n-1, idx, val);
```

```
    }
  }

  return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Meera is working on a text processing task where she wants to identify the longest common prefix among a given set of words. She decides to use a Trie structure to efficiently compute this prefix.

Write a program to help Meera insert the given words into a Trie and then determine the longest common prefix among them.

Input Format

The first line contains an integer n, representing the number of words.

The next n lines each contain a single word.

Output Format

The output prints "The longest common prefix is: <prefix>" if a common prefix exists.

Otherwise, print "There is no common prefix"

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
aeroplane
aerospace
aerial
aesthetic

Output: The longest common prefix is: ae

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

int main() {
    int n;
    scanf("%d", &n);

    char words[10][100];

    for (int i = 0; i < n; i++) {
        scanf("%s", words[i]);
    }

    char prefix[100];
    strcpy(prefix, words[0]);

    for (int i = 1; i < n; i++) {
        int j = 0;

        while (prefix[j] && words[i][j] && prefix[j] == words[i][j]) {
            j++;
        }

        prefix[j] = '\0'; // shorten prefix
    }

    if (strlen(prefix) > 0) {
        printf("The longest common prefix is: %s", prefix);
    } else {
        printf("There is no common prefix");
    }

    return 0;
}
```

Status : Correct ***Marks : 10/10***